

Circuite de memorare RAM și ROM și sisteme cu circuite de memorare

Conținut laborator:

- 1. Obiectivul lucrării.**
- 2. Noțiuni teoretice:**
 - a. Aspecte generale despre circuitele de memorie**
 - b. Circuite de memorare construite în laborator**
 - c. Proiectarea sistemelor cu memorii**
- 3. Cerințe laborator.**
- 4. Temă.**

Obiectivul lucrării:

În acest laborator sunt prezentate în primul rând modurile în care sunt comandate circuitele de memorie. Așa cum s-a prezentat și în laboratorul anterior, alături de semnalele de adresare și cele de date sunt necesare semnale de comandă scriere și citire. Pe lângă acestea mai este necesar și un semnal de validare a funcționării întregului circuit necesar conectării acestuia pe o magistrală comună.

Partea a doua a laboratorului este dedicată conectării mai multor circuite de memorare pe o magistrala comună. Schemele construite aici sunt exemple practice, la scară redusă, de sisteme cu circuite de memorare. Laboratorul servește la aprofundarea noțiunilor legate de lucru cu circuitele de memorie și de modul în care acestea pot fi integrate în sistem.

Noțiuni teoretice:

A. Aspecte generale despre circuitele de memorie:

Circuitele de memorare se împart în două mari clase : memorii ROM și memorii RAM. Denumirea de ROM provine din limba engleză Read Only Memory – memorie numai pentru citit și se referă la circuitele care în regim normal de funcționare pot să fie numai citite. Denumirea de RAM provine din Random Acces Memory – memorie cu acces aleator și se referă la circuitele care în regim normal de funcționare pot fi atât citite cât și scrise. Trăsături comune celor 2 memorii:

- liniile de selecție sunt denumite linii de adresă. Notăția acestor linii este generalizată, $A_0, \dots, A_{n-1}$, A_0 -linia cel mai puțin semnificativă;

- informațiile ce sunt enumerate se preiau din circuitele de memorie, sunt transferate prin intermediul liniilor de date. Majoritatea circuitelor de memorie (RAM) au liniile pentru datele de intrare comune cu datele de ieșire.

Celula elementară într-o locație de memorie se numește bit. Majoritatea circuitelor de memorie, în momentul aplicării unei informații de adresă selectează simultan mai multe celule elementare de memorie. Acestea formează o locație de memorie. Practic, circuitele de memorie realizate de diferite firme producătoare conțin valori de tipul 1,2,4,8,12,16,32 biți în codul unei locații de memorie.

Pentru câteva combinații de biți într-o locație de memorie există denumiri specifice: 8 biți=1 octet=1 byte; 16 biți=1 word; 4 biți=1 nibble. Conținutul unei locații

de memorie se transferă către exteriorul circuitului de memorie prin intermediul liniilor de comandă acestea fiind notate $D_0, D_1, \dots$. D_0 =bitul cel mai puțin semnificativ. Se numește organizarea unui circuit de memorie produsul neefectuat între numărul locațiilor de memorie și dimensiunea în biți a unei locații. Numărul locațiilor de memorie e dat de $2^{\text{nr. linii adresă}}$ pentru circuitul respectiv. De exemplu: $2^{10 \text{ locații de memorie}} = 1024 = 1K$; $2^{20} = 1M$; $2^{30} = 1G$. O memorie cu 14 linii de adresă și 8 biți într-o locație : $A_0, \dots, A_{13}$ - linii de date; $D_0, \dots, D_7$ - linii de adresă; $12^{14} \cdot 8 = 2^4 \cdot \underbrace{2^{10}}_K \cdot 8 = 2k \cdot 8 = 16k \cdot 8 = 16K_0 \leftarrow \text{octet}$.

Se numește capacitatea unui circuit de memorie și se exprimă în biți produsul efectuat între numărul locațiilor de memorie și numărul de biți aflați într-o locație. $16K \cdot 8 = 128Kb$

1) Circuite de memorare de tip ROM:

Se pot clasifica în mai multe categorii:

- OTP (one time programing)
- EPROM (erasable ROM), care se pot șterge, doar de tip unipolar, prezintă fereastră de quartz;
- EEPROM (electrically erasable PROM), cu acces paralel=flash, admit 1 mil. $\rightarrow$ 10 mil. cicluri scriere-citire.

La circuitele ROM bipolare arderea fuzibilă pentru o celulă de memorie realizează procesul de memorare. La o memorie ROM unipolară memorarea se realizează prin stocarea sarcinii electrice într-o grilă fixă și una flotantă a unui tranzistor MOS. Descărcarea acestei sarcini electrice are loc prin expunerea pastilei de siliciu la raze ultraviolete. La o memorie ROM-OTP aceasta nu e posibil deoarece memoria nu dispune de o fereastră de quartz pentru expunere. Un circuit ROM conține următoarele linii de semnal:

- linii de adrese
 - linii de date
- } numărul lor e dat de organizarea circuitului

Întotdeauna un circuit conține:

- linia $\overline{CE}$ (chip enable);
- $\overline{OE}$ (output enable).

Etajele de ieșire la o memorie ROM sunt de tip tristate. La memoriile ROM bipolare putem avea ieșiri:

- open collector;
- tristate.

Intrarea $\overline{CE}$ la activare permite adresarea și selecția locației de memorie cu care se lucrează. Semnalul $\overline{OE}$ la activare deschide etajele de ieșire cu liniile de date ale circuitului.

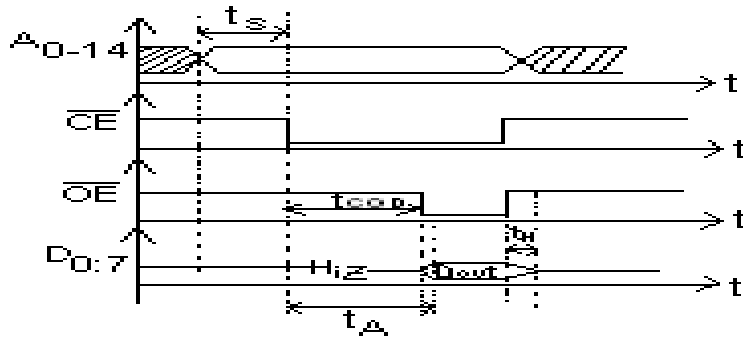


Fig.1. Formele de undă pentru accesarea unei memorii

2) Circuite de memorare de tip RAM:

Memoriile RAM se împart în următoarele categorii: statice SRAM, dinamice DRAM. Pentru o memorie RAM-static celula de memorare e clasicul latch.

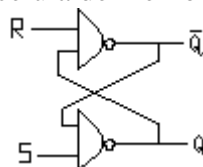


Fig.2. Celula de memorare SRAM

Pentru o memorie RAM-dinamic:

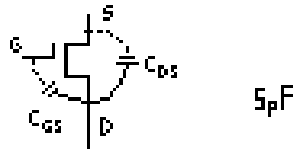


Fig.3. Celula de memorare DRAM

Capacitățile drenă-sursă și grilă-sursă vor reprezenta mijlocul de stocare, sub forma unor câmpuri electrice. La memoria DRAM capacitatea parazită care servește în cadrul procesului de memorare auxiliară unui tranzistor unipolar are tendința ca foarte repede să-și piardă sarcina electrică stocată între armături. Din acest motiv e nevoie ca la intervale relativ mici să se intervină asupra structurii prin reîmprospătarea sarcinii electrice stocate. Procesul poartă denumirea de refresh și are loc printr-o operație de citire. Memoriile dinamice au avantajele unor capacități mari de memorare integrate pe aceeași capsule de siliciu la consumuri energetice relativ reduse. Costul unei memorii DRAM e mult mai mic decât al unei memorii statice RAM având aceeași capacitate.

B. Circuite de memorare construite în laborator

Pentru exemplificare în acest laborator s-au realizat mai multe circuite de memorare. În primul rând este vorba de o memorie SRAM cu organizarea 16x8.

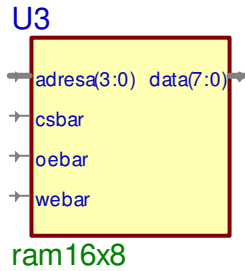


Fig.4.RAM 16x8

Această memorie, realizată în cadrul laboratorului, are următoarele caracteristici:

Memoria ram16x8

Are dimensiunea de 16 octeți.

Intrări:

Adresa(3:0) – intrari adrese – 4 biți;

Cs_bar – intrare pentru selectarea circuit – 1 bit;

Oe_bar – intrare pentru selectarea ieșirii – 1 bit;

We_bar – intrare pentru validarea scrierii – 1 bit.

Intrări-ieșiri:

Data (7:0) – intrare-ieșire de date – 8 biți.

În limbajul VHDL, descrierea unei memorii RAM 16x8 se face ca în listingul de mai jos:

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.all;
```

```
entity ram16x8 is
```

```
    port(
```

```
        csbar : in STD_LOGIC;
```

```
        oebar : in STD_LOGIC;
```

```
        webar : in STD_LOGIC;
```

```
        adresa : in STD_LOGIC_VECTOR(3 downto 0);
```

```
        data : inout STD_LOGIC_VECTOR(7 downto 0)
```

```
    );
```

```
end ram16x8;
```

```
architecture ram16x8_a of ram16x8 is
```

```
    type memorie is array (0 to 15) of std_logic_vector(7 downto 0);
```

```
    begin
```

```
        process(csbar,oebar,webar,adresa)
```

```
            variable data_in : memorie;
```

```
            variable index : integer;
```

```
            begin
```

```
                if csbar = '0' then
```

```
                    index := 0;
```

```

for i in 0 to 3 loop
    if adresa(i)='1' then
        index := index + 2**i;
    end if;
end loop;
if oebar = '0' then
    data <= data_in(index);
else
    if webar = '0' then
        data_in(index) := data;
    else
        data <= "ZZZZZZZZ";
    end if;
end if;
else
    data <= "ZZZZZZZZ";
end if;
end process;

end ram16x8_a;

```

Semnalele de comandă (cs_bar, oe_bar și we_bar) sunt semnale active 0 logic. Deci, când aceste semnale sunt în 1 logic înseamnă că funcțiile comandate sunt dezactivate. Secvența de scriere pentru acest circuit este următoarea:

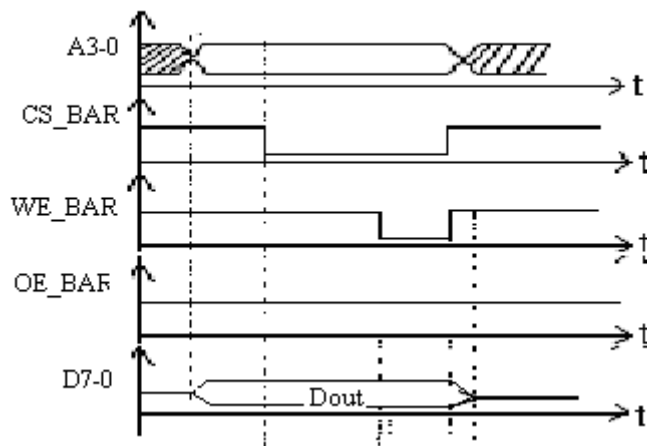


Fig.5. Secvența de semnale pentru operația de scriere

Pentru citire secvența este următoarea:

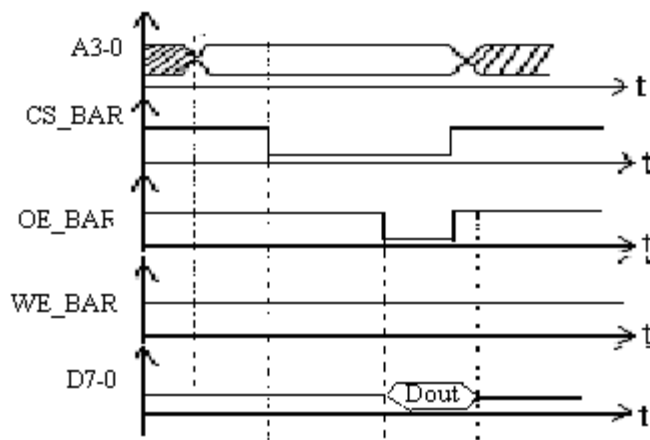


Fig.6. Secvența de semnale pentru operația de citire

Ca la orice circuit de memorare secvența de scriere, ca și cea de citire, începe cu transmiterea adresei locației. La secvența de scriere este transmisă și data care se dorește scrisă. Urmează apoi trecerea în 0 a semnalului de selectarea a circuitului apoi comanda operației care se dorește să se efectueze. Așa cum se observă și în formele de undă prezentate în figurile de mai sus, fiecare din pașii prezentați mai sus necesită un anumit interval de timp. Pentru circuitele întâlnite în practică acesta are valori clar precizate în formele de undă din datele de catalog. Justificarea ordinii aplicării semnalelor și a necesității timpilor a fost făcută în laboratorul anterior!

Un alt circuit construit și utilizat în acest laborator este o memorie ROM cu organizarea 256x8.

Circuitul prezentat în figura de mai jos are următoarele caracteristici:

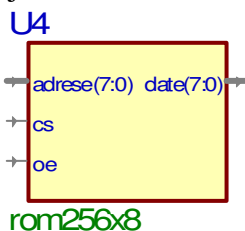


Fig.7. ROM 256x8

Memoria ram16x8

Are dimensiunea de 16 octeți.

Intrări:

Adrese(7:0) – intrari adrese – 8 biți;

Cs – intrare pentru selectarea circuit – 1 bit;

Oe – intrare pentru selectarea ieșirii – 1 bit;

Intrări-ieșiri:

Date (7:0) – ieșire de date – 8 biți.

Descrierea memoriei ROM echivalează cu cea a unui multiplexor de date și a unui buffer tree – state, așa cum este prezentat în listingul de mai jos:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity rom256x8 is
    port(
        cs : in STD_LOGIC;
        oe : in STD_LOGIC;
        adrese : in STD_LOGIC_VECTOR(7 downto 0);
        date : out STD_LOGIC_VECTOR(7 downto 0)
    );
end rom256x8;

architecture rom256x8_a of rom256x8 is
    signal data_in : std_logic_vector(7 downto 0);
    begin

        with adrese select
            data_in <=

                --aici urmeaza continutul memoriei ROM
                --data          adresa
                X"78" when X"00",
                X"20" when X"01",
                X"FF" when others;

        process(cs,oe,data_in)
        begin
            if cs='0' and oe = '0' then
                date <= data_in;
            else
                date <= "ZZZZZZZZ";
            end if;
        end process;

    end rom256x8_a;

```

Memoria ROM este privită aici ca un circuit care permite numai citiri fără scrieri. Acesta este modul de operare normal al memoriei ROM.

În practică, scrierea unei memorii din clasa ROM se face într-o operație denumită programare. Deci programarea este un regim separat de cel de operare normal. Aceasta deoarece programarea presupunea la circuitele mai vechi prezența unor tensiuni suplimentare de programare. La memoriile ROM noi, cum ar fi cele din clasa FLASH sau EEPROM, regimul de programare presupune aplicarea unor secvențe speciale de scriere, având însă aceleași tensiuni de operare ca în regimul de operare normal (numai citire).

De orice tip ar fi ele, memoriile din clasa ROM au două regimuri distincte de funcționare: regimul normal – numai citire și regimul de programare – scriere. În acest laborator se va lucra cu o memorie ROM cu organizarea 256x8 numai în regimul normal de operare, memoria având deja scris în ea anumite date. În laboratorul următor, prin intermediul unei aplicații, se va realiza și regimul de programare.

În regimul normal de operare, secvența parcursă este numai una de citire, similară cu citirea de la memoriile din clasa SRAM.

Într-un sistem pe care rulează un program, constantele declarate în program sunt stocate într-o memorie nevolatilă, din clasa ROM iar variabilele într-o memorie din clasa RAM. Așa cum o să se vadă în laboratoarele următoare însăși secvența de program constituie de fapt o informație care trebuie stocată într-o memorie nevolatilă!

C. Proiectarea sistemelor cu memorii

Un sistem de calcul, alături de microprocesor, trebuie să aibă și circuite de memorare, pentru stocarea variabilelor, a constantelor și a secvențelor de program.

Arhitectura unui sistem de calcul tipic este prezentată în figura de mai jos.

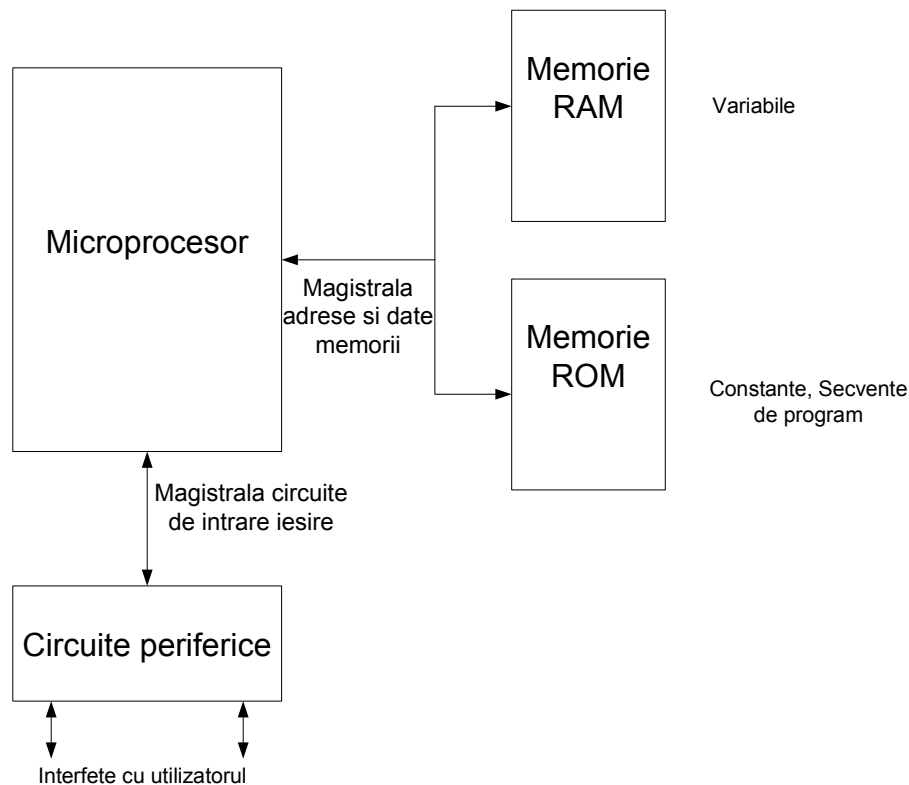


Fig.8. Schema bloc a unui sistem cu microprocesor

Accesarea memoriilor RAM, ROM dar și a circuitelor periferice se face prin intermediul unor adrese. La nivelul sistemului cu microprocesor se poate defini noțiunea de spațiu de adresare, care constă în totalitatea locațiilor care le poate adresa.

Pentru memoria din figura 4 spațiul de adresare este de la 00h până la 0fh deci un total de 16 locații distincte.

În cele ce urmează se va pune problema, pentru exemplificare, a proiectării unei memorii cu organizarea de 32x8. Memoria rezultată va trebui să ocupe un spațiu de adresare pentru 32 locații începând cu adresa 00h.

Problema 1. Construirea unei memorii SRAM 32x8 folosind memorii SRAM16x8 și plasarea ei într-un spațiu de adresare începând cu adresa 0.

În cazul în care se dorește construirea unei memorii cu dimensiunea de 32 locații, spațiul de adresare va fi, în mod evident cuprins între adresele 00h și 1fh. Dimensiunea spațiului de adresare este întotdeauna multiplu de 2.

Construcția memoriei 32x8 pornește de la un calcul referitor la numărul de circuite de memorie 16x8 utilizate. Aici avem:

$$\frac{32 \times 8}{16 \times 8} = 2 \times 1 = 2 \text{ circuite .}$$

Deci sunt necesare două circuite de memorie 16x8 pentru construirea unei memorii 32x8. Pasul următor îl constituie construirea spațiului de adresare. Fiind vorba de o memorie de 32 de locații sunt necesari 5 biți de adresă care să fie folosiți. Spațiul de adresare trebuie să fie unul continuu și trebuie evitată adresarea duală.

Adresa de început este 00h. Deci harta de adrese va fi următoarea:

	1	1	1	1		
A4	A3	A2	A1	A0	Hexa	Circuit
0	0	0	0	0	00h	RAM1
0	1	1	1	1	0fh	
1	0	0	0	0	10h	RAM2
1	1	1	1	1	11h	

Cel de-al 4-lea bit de adrese (A4) va fi folosit pentru selectarea circuitului de memorie prin comandarea semnalului de selecție circuit CS_BAR.

Pentru construirea unei memorii de 32x8 folosind memorii 16x8 se folosește o schemă prin care sunt selectate separat cele două memorii, printr-un demultiplexor 1:2 așa cum se vede în figura de mai jos (figura 7).

Selecția globală este dată de intrarea de activare a circuitului demultiplexor în timp ce intrările de comandă scriere globală ca și cea de comandă citire sunt conectate la intrările de comandă scriere și citire ale memoriilor.

Magistrala de date poate fi comună, deoarece la un moment dat este activat un singur circuit.

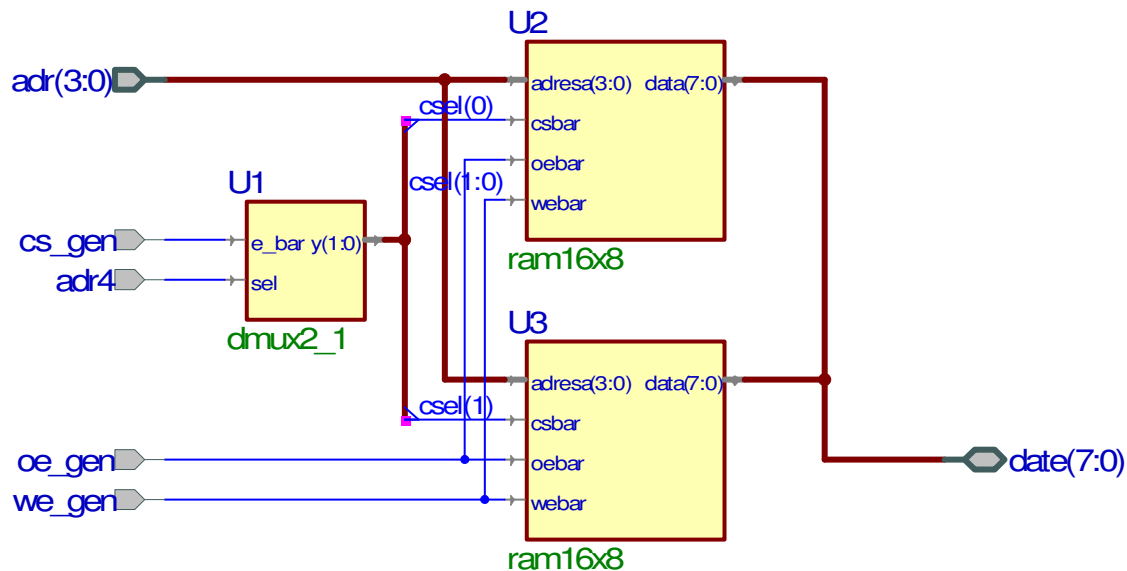


Fig.9. Memoria 32x8 realizată din memorii 16x8

Formele de undă rezultate în urma simulării circuitului sunt prezentate în figura de mai jos.

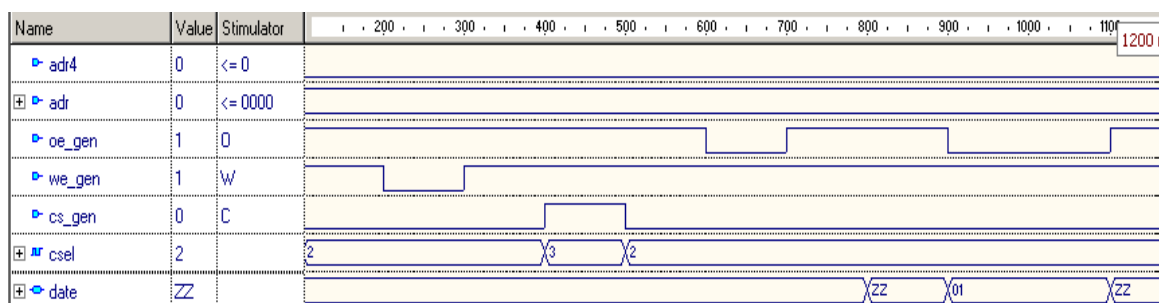


Fig.10. Formele de undă la scrierea și citirea din circuitul de memorie 32x8 proiectat

Spațiul de adresare pentru un sistem de calcul este unul continuu și începe, în mod normal, de la adresa 0. În același spațiu de adresare se află, de regulă, mai multe circuite de memorare, unele chiar de mai multe tipuri.

Problema 2. Construirea unui circuit cu memorii ROM și RAM și plasarea acestuia într-un spațiu de adresare continuu începând cu adresa 0.

În cele ce urmează se cere realizarea unui circuit, accesat de către un microprocesor, care să conțină următoarele:

- o memorie ROM cu organizarea 256x8 plasată începând cu adresa 0000h;
- o memorie SRAM cu organizarea 32x8, alcătuită din două memorii SRAM cu organizarea 16x8;
- spațiul de adresare trebuie să fie continuu și să se evite adresarea duală.

Înainte de a porni la construirea circuitului propriu zis se vor face niște calcule legate de logica de adresare.

Pentru memoria ROM condițiile impuse sunt legate de adresa de început.
Harta de adrese este următoarea:

	1	1	1	1	1	1	1	1		Max ROM
					1	1	1	1		Max RAM
A8	A7	A6	A5	A4	A3	A2	A1	A0	Hexa	Circuit
0	0	0	0	0	0	0	0	0	000h	ROM
0	1	1	1	1	1	1	1	1	0ffh	
1	0	0	0	0	0	0	0	0	100h	RAM1
1	0	0	0	0	1	1	1	1	10fh	
1	0	0	0	1	0	0	0	0	110h	RAM2
1	0	0	0	1	1	1	1	1	11fh	
adr 8	adr 7	adr 6	adr 5	adr 4	adr 3	adr 2	adr 1	adr 0		
S1				S0						Selectii DMUX

Pentru selecția circuitelor se vor folosi doi pini, pinul al 9-lea de adrese și pinul 4. Pinul 4 este utilizat pentru selecția circuitelor RAM1 sau RAM0 atunci când memoria ROM nu este selectată (deci pentru adrese mai mari de 0ffh).

Schema de conectare ale memoriilor pentru spațiul comun de adrese este prezentată în figura de mai jos.

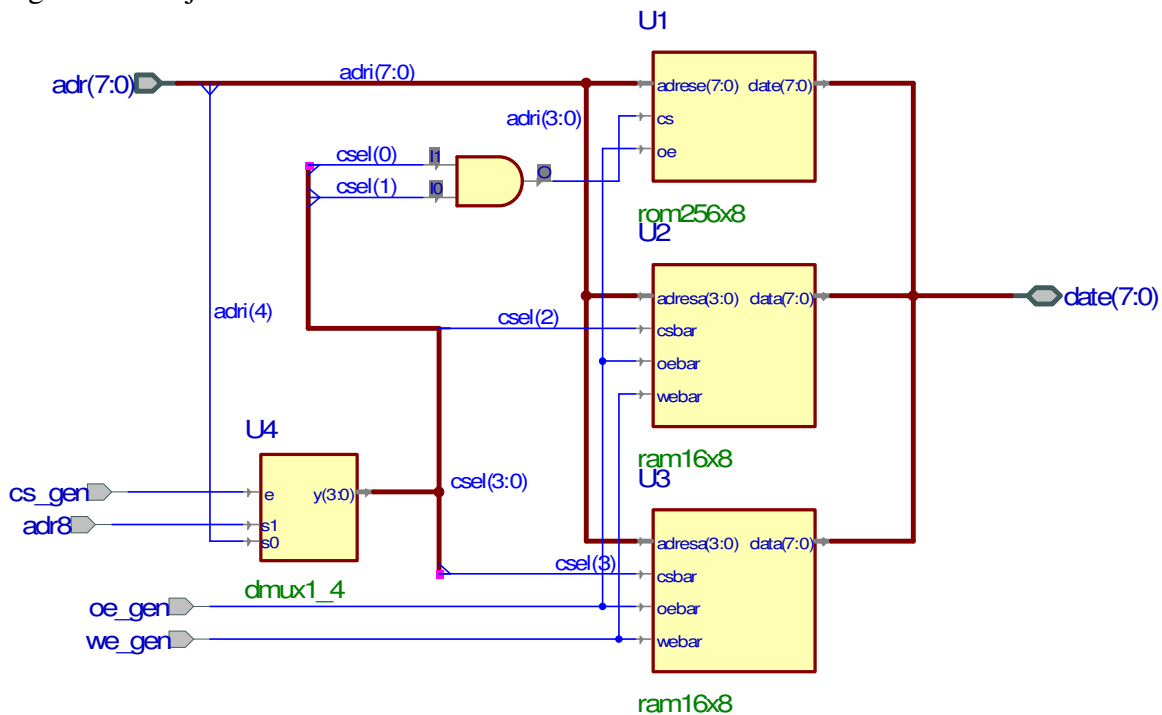
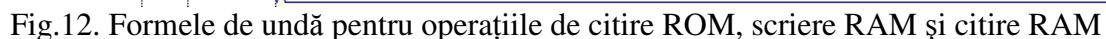


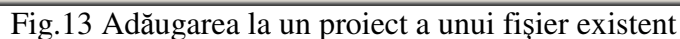
Fig.11. Conectarea a 3 memorii 2SRAM și una ROM la un spațiu de adrese continuu

Formele de undă pentru operații de citire din memoria ROM, scriere și citire din memoria RAM sunt prezentate în figura de mai jos.



1. Lucrul cu memoria SRAM 16x8

- se va crea un proiect, urmărind etapele prezentate în laboratoarele anterioare;
- se va adăuga la proiectul existent fișierul memorii.vhd aflat în directorul cu laboratorul curent (selecție Add new file apoi Add existing file fig.13);
- se va compila fișierul adăugat;
- se va crea un fișier schematic gol;
- se vor adăuga componente din proiect (Units without symbols)
- se va construi circuitul alcătuit din memoria din figura 4. Se va compila.



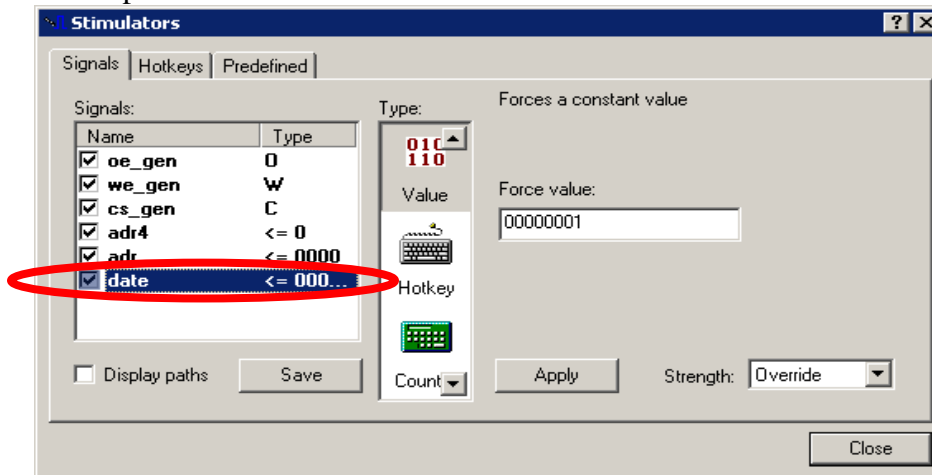
- | | | |
|--------|----------|-------------|
| Semnal | Stimul | Valoare |
| Adr | Value | 0000...1111 |
| Oe_bar | Hotkey O | 0,1 |
| We_bar | Hotkey W | 0,1 |

Cs_bar	Hotkey C	0,1
Date	Value	00000000...11111111

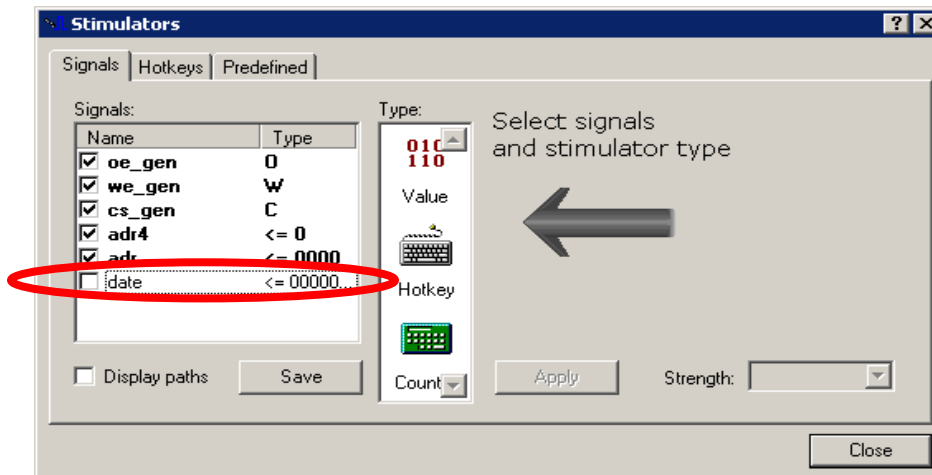
- se vor efectua 3 operații de scriere și citire de la locații diferite, ținând cont de formele de undă din figurile 5 și 6.

Notă: Atribuirea stimulilor pe semnalul de date se face ținând cont de faptul că magistrala „Date” este una bidirecțională. Astfel, atunci când se efectuează operația de scriere, se va plasa stimulul pe intrare, ca în figura de mai jos.

Se va urmări ca plasarea stimulului-ului să fie bifată în listă.



Când se efectuează o operație de citire se va urmări scoaterea stimulului din listă și apoi rularea unui pas în simulare. Se va urmări debifarea din listă. Se va apăsa butonul Close (fără Apply!!)



Se vor efectua cel puțin 3 operații de scriere, în locații distincte, și 3 operații de citire ale memoriei pentru punctarea acestui exercițiu.

2. Lucrul cu memoria ROM 256x8

Pentru lucrul cu memoria ROM se vor parcurge pașii:

- se va adăuga la proiectul existent fișierul rom256x8.vhd aflat în directorul cu laboratorul curent (fig.13);
- se va compila fișierul adăugat;
- se va crea un fișier schematic gol;
- se vor adăuga componente din proiect (Units without symbols)
- se va construi circuitul alcătuit din memoria din figura 7. Se va compila.
- se va realiza simularea memoriei cu următorii stimuli pe pini:

Semnal	Stimul	Valoare
Adr	Value	00000000...11111111
Oe	Hotkey O	0,1
Cs	Hotkey C	0,1

- se vor realiza 3 operații de citire de la locații diferite;
- se va realiza implementarea circuitului pe machetă cu următoarele conexiuni:

Semnal	Stimul	Valoare
Adr	SW_DIP0..7	00000000...11111111
Oe	SW_USER0	0,1
Cs	SW_USER1	0,1

3. Construirea circuitului de memorie cu organizarea 32x8 folosind memorii 16x8
Pentru construirea circuitului de memorie cu organizarea 32x8 folosind circuite 16x8 se va proceda în felul următor:

- Se va crea, urmărind pașii descriși în laboratorul anterior, un fișier VHDL, în care se va descrie demultiplexorul pentru selecții.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity dmux2_1 is
    port(
        e_bar : in STD_LOGIC;
        sel : in STD_LOGIC;
        y : out STD_LOGIC_VECTOR(1 downto 0)
    );
end dmux2_1;

architecture dmux2_1a of dmux2_1 is
begin

    process(e_bar,sel)
    begin
        if e_bar = '0' then
            case sel is
                when '0' => y <= "10";
                when '1' => y <= "01";
                when others => y <= "11";
            end case;
        else
            y <= "11";
        end if;
    end process;
end dmux2_1a;
```

```

end process;

end dmux2_1a;

```

- Se va compila fișierul.
- Se va crea un fișier schematic și se va introduce schema din figura 9.
- Se va efectua simularea circuitului plasând următorii stimuli:

Semnal	Stimul	Valoare
Adr4	Value	0,1
Adr	Value	0000...1111
Oe_gen	Hotkey O	0,1
We_gen	Hotkey W	0,1
Cs_gen	Hotkey C	0,1
Date	Value	00000000...11111111

- Se vor realiza 3 operații de scriere și 3 operații de citire de la adrese diferite pentru a se evidenția modul în care lucrează memoria.

4. Construirea circuitului cu memorie ROM și două memorii RAM, în spațiul de adresare continuu.

Pentru construirea circuitului de memorie cu organizarea 32x8 folosind circuite 16x8 se va proceda în felul următor:

- Se va crea, urmărind pașii descriși în laboratorul anterior, un fișier VHDL, în care se va descrie demultiplexorul pentru selecții.

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity dmux1_4 is
    port(
        e : in STD_LOGIC;
        s1 : in STD_LOGIC;
        s0 : in STD_LOGIC;
        y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end dmux1_4;

architecture dmux1_4_a of dmux1_4 is
    signal sint : std_logic_vector(1 downto 0);
begin

    sint(0)<=s0;
    sint(1)<=s1;
    process(e,sint)
    begin
        if e = '0' then
            case sint is
                when "00" => y <= "1110";

```

```

        when "01" => y <= "1101";
        when "10" => y <= "1011";
        when "11" => y <= "0111";
        when others => y <= "1111";
    end case;
else
    y <= "1111";
end if;
end process;
end dmux1_4_a;

```

- Se va compila fișierul.
- Se va crea un fișier schematic și se va introduce schema din figura 11.
- Se va efectua simularea circuitului plasând următorii stimuli:

Semnal	Stimul	Valoare
Adr8	Value	0,1
Adr	Value	00000000...11111111
Oe_gen	Hotkey O	0,1
We_gen	Hotkey W	0,1
Cs_gen	Hotkey C	0,1
Date	Value	00000000...11111111

- Se vor realiza: 6 operații de citire din spațiul memoriei ROM și 3 operații de scriere și 3 operații de citire de la adrese diferite în spațiul memoriilor RAM pentru a se evidenția modul în care lucrează memoria.

Tema

5. Să se proiecteze un circuit de memorie SRAM 64x8 folosind memorii 16x8, urmărind pașii de la punctele anterioare.

6. Să se realizeze un circuit aflat într-un spațiu de adresare cu următoarea distribuție:

Circuit	Adresa început
SRAM0 16x8	Adresa 00h
SRAM1 16x8	Adresa 0fh
ROM 256x8	Adresa 100h

Criterii de evaluare:

1	1P
2	2P
3	2P
4	2P
5	3P
6	3P